Reporte de Auditoría Técnica Exhaustiva

Proyecto: Inventario TravelNet (Frontend Angular & Backend Express)

Fecha de auditoría	31 de octubre de 2026
Directorio auditado	`/var/www/html/Inventario_Travelnet`
Stack Tecnológico	Angular 21.2 (Feature-Based Architecture, TypeScript 5.9, Dexie IndexedDB) | Node.js + Express 5.3 (JavaScript CommonJS, MariaDB 3NF, JWT, Bcrypt)
Alcance	Arquitectura Frontend/Backend, Detección de Errores, Comunicación API REST, Seguridad, Calidad/Mantenibilidad y Rendimiento.

1. Resumen General del Estado del Proyecto

El proyecto **Inventario TravelNet** cuenta con una base funcional operativa sobre el esquema normalizado en Tercera Forma Normal (**3NF**) en MariaDB. El Backend separa los privilegios de base de datos mediante pools de conexión por rol (`mysql_login`, `mysql_admin`, `mysql_mostrador` y `mysql_instalador`), emplea consultas parametrizadas (`?`) que previenen inyección SQL, almacena contraseñas con hashes `bcrypt` y el Frontend en Angular 21 compila sin errores de TypeScript e incluye soporte de almacenamiento offline con IndexedDB (Dexie).

No obstante, la auditoría identificó **13 errores confirmados** (2 críticos, 5 altos, 5 medios y 1 bajo) y **3 problemas estructurales de arquitectura/mantenibilidad** que deben corregirse prioritariamente antes de un despliegue productivo. Los riesgos más importantes se concentran en la protección de archivos con credenciales, exposición sin autenticación de fotografías de evidencia con datos GPS, vulnerabilidad DOM XSS al generar tickets de impresión, validaciones de autorización ejecutadas únicamente en el navegador y desfase de zona horaria UTC en el cálculo del corte de caja.

2. Análisis de Arquitectura

2.1 Arquitectura Frontend (Angular — Feature-Based Architecture)

- **Organización por funcionalidades:** El proyecto divide sus pantallas en `Frontend/src/app/features/` (`cliente`, `corte-caja`, `home`, `imagen_instalacion`, `instalacion`, `mensualidad`, `OLT`, `pago`, `plan`, `servicios`, `Torre`, `usuario`) usando componentes *standalone* y carga diferida (`loadChildren`).
- **Ausencia de capa `core/`:** No existe un directorio `src/app/core/`. El guard de autenticación (`shared/auth.guard.ts`), el interceptor HTTP (`shared/interceptors/auth.interceptor.ts`) y la instancia singleton de IndexedDB (`db/app.db.ts`) pertenecen conceptualmente a `core/`.
- **Duplicación de modelos e interfaces entre features:**
  - `olt.model.ts` y `torre.model.ts` están triplicados en `features/OLT/model/`, `features/cliente/models/` y `features/instalacion/models/`.

    - `cliente.model.ts` está duplicado en `features/cliente/models/` y `features/instalacion/models/`.
    - `corte-caja.model.ts` está duplicado en `features/corte-caja/model/` y `features/pago/models/`.
    - `imagen_instalacion.model.ts` y `usuario.model.ts` están duplicados dentro de `features/cliente/models/`.
- **Acoplamiento cruzado entre funcionalidades:**
    - `SyncService` está ubicado dentro de `features/pago/services/sync.service.ts`, pero sincroniza clientes, instalaciones y pagos, y es inyectado en el componente raíz `App` (`src/app/app.ts`).
    - `ListaPago` importa directamente el componente de página `ListaMensualidad` (`features/mensualidad/pages/lista-mensualidad/lista-mensualidad.ts`).
    - `Login` reside dentro de `features/usuario/pages/login/login.ts` y se importa de forma ansiosa (*eager loading*) en `app.routes.ts`.
- **Feature sin implementar (código muerto):** El directorio `features/servicios/` contiene un servicio y un componente vacíos sin lógica ni uso real.

2.2 Arquitectura Backend (Node.js + Express)

- **Separación de capas:** Estructura dividida en `routes/`, `src/controller/`, `src/model/` y `src/config/`.
- **Ausencia de capa de validación de entrada (DTO/Schema):** Los controladores pasan `req.body` directamente a los modelos sin validar tipos ni estructura.
- **Almacenamiento físico en controlador:** `imagen_InstalacionController.create` decodifica Base64 y escribe archivos en disco directamente desde el controlador.
- **Manejo de errores inconsistente:** Cada controlador repite bloques `try/catch` devolviendo `res.status(500).json({ error: error.message })`, mientras que el manejador global de errores en `Backend/app.js` intenta renderizar una plantilla HTML Pug (`res.render('error')`) en lugar de retornar JSON.

3. Resumen Estadístico de Hallazgos

Categoría	Errores Confirmados	Recomendaciones / Mejoras	Total
Errores de seguridad	5 (`AUD-001` a `AUD-005`)	0	**5**
Errores funcionales	3 (`AUD-006`, `AUD-007`, `AUD-009`)	0	**3**
Errores de comunicación API	2 (`AUD-008`, `AUD-010`)	0	**2**

Categoría	Errores Confirmados	Recomendaciones / Mejoras	Total
Errores arquitectónicos	1 (`AUD-011`)	2 (Estructura `core/` y servicios)	**3**
Problemas de rendimiento	1 (`AUD-012`)	0	**1**
Problemas de mantenibilidad / Estándares	1 (`AUD-013`)	1 (Consolidación de modelos)	**2**
TOTAL	**13**	**3**	**16**

ID	Gravedad	Categoría	Archivo Principal	Resumen del Problema
AUD-001	**CRÍTICO**	Seguridad	`.gitignore`, `contraseñas.txt`	Archivos con contraseñas en texto plano y `Backend/.env` no ignorados en Git.
AUD-002	**CRÍTICO**	Seguridad	`Backend/src/config/jwt.js`	`getSecret()` usa cadena vacía `''` como fallback si falta `JWT_SECRET`.
AUD-003	**ALTO**	Seguridad / Rendimiento	`Backend/app.js`, `imagen_InstalacionController.js`	Imágenes con GPS servidas sin autenticación en `/uploads` y escritura síncrona bloqueante.
AUD-004	**ALTO**	Seguridad (XSS)	`lista-pago.ts`, `insertar-pago.ts`	DOM XSS mediante `document.write()` sin escapar datos en tickets de pago y corte.
AUD-005	**ALTO**	Seguridad / Autorización	`Backend/app.js`, `lista-pago.ts`	No se persiste `req.user` del JWT; autorización de corte por Admin validada solo en Frontend.
AUD-0	**ALTO**	Funcional	`lista-pago.ts`, `home.ts`	Corte de caja usa

ID	Gravedad	Categoría	Archivo Principal	Resumen del Problema
06				fecha UTC (`toISOString()`), omite pagos múltiples y no excluye pagos ya cortados.
AUD-007	**ALTO**	Funcional	`actualizar-instalacion.ts`, `instalacionController.js`	Se fuerza `TarifaId = null` al actualizar instalación y se reescriben columnas innecesariamente.
AUD-008	**MEDIO**	Comunicación API	Servicios en `Frontend/src/app/features/**`	Inconsistencia de mayúsculas en endpoints y `http://localhost:3000` hardcodeado en 12 servicios.
AUD-009	**MEDIO**	Funcional / BD	`instalacion.js`, `Imagen_Instalacion.js`	Falta validación de idempotencia para `Uuid_local` al crear instalaciones e imágenes.
AUD-010	**MEDIO**	Comunicación API / Seguridad	`auth.guard.ts`, `auth.interceptor.ts`	`authGuard` no verifica expiración del JWT y el interceptor no captura errores HTTP `401`.
AUD-011	**MEDIO**	Arquitectura / API	`Backend/app.js`, Controladores	Error handler global devuelve HTML en vez de JSON; controladores retornan siempre `500` con `error.message`.
AUD-012	**MEDIO**	Rendimiento	`home.ts`, `lista-pago.ts`, `sync.service.ts`	Descarga de 6 tablas completas en `Home`, llamadas manuales a `ngOnInit()` y falta de

ID	Gravedad	Categoría	Archivo Principal	Resumen del Problema
				mutex en `SyncService`.
AUD-013	**BAJO**	Mantenibilidad	`package.json`, `routes/Usuario.js`	Paquete erróneo `"corns"`, `"jsonwebtoken"` sin uso en Frontend e importaciones/funciones muertas.

4. Detalle Exhaustivo de Problemas Detectados

AUD-001 — Archivos con contraseñas en texto plano y .gitignore incompleto

1. **Identificador del problema:** `AUD-001`
2. **Archivo donde se encuentra:** `/var/www/html/Inventario_Travelnet/.gitignore`, `/var/www/html/Inventario_Travelnet/contraseñas.txt` y `/var/www/html/Inventario_Travelnet/PLAN_INSERTS_HASH.txt`
3. **Línea aproximada del error:** Líneas 16–30 de `.gitignore`
4. **Categoría del problema:** Error de seguridad (Error confirmado)
5. **Nivel de gravedad:** **CRÍTICO**
6. **Descripción del problema:** Los archivos `contraseñas.txt` y `PLAN_INSERTS_HASH.txt` contienen contraseñas reales de los usuarios de MariaDB y de usuarios de la aplicación en texto plano y no están incluidos en `.gitignore`. Adicionalmente, la regla `/.env` en la línea 16 de `.gitignore` lleva una barra inicial, lo que en Git solo ignora un archivo `.env` en la raíz del repositorio pero **no ignora** `Backend/.env`.
7. **Explicación de por qué representa un problema:** Cualquier confirmación de cambios (`git add . && git commit`) subirá las credenciales de producción/desarrollo al historial del repositorio Git.
8. **Consecuencias que podría ocasionar:** Exposición total de las cuentas de base de datos (`mysql_admin`, `mysql_mostrador`, `mysql_instalador`, `mysql_login`) y del secreto de firma JWT.
9. **Solución recomendada:** Incluir `**/.env`, `contraseñas.txt` y `PLAN_INSERTS_HASH.txt` en `.gitignore`.
10. **Ejemplo de código corregido (`.gitignore`):**

```
# Environment and credentials
files
.env
.env.*
**/.env
```

credenciales.txt
contraseñas.txt
PLAN_REMEDIACION.txt
PLAN_INSERTS_HASH.txt

AUD-002 — Secreto JWT con fallback a cadena vacía ('')

1. **Identificador del problema:** `AUD-002`
2. **Archivo donde se encuentra:** `/var/www/html/Inventario_Travelnet/Backend/src/config/jwt.js`
3. **Línea aproximada del error:** Línea 8
4. **Categoría del problema:** Error de seguridad (Error confirmado)
5. **Nivel de gravedad:** **CRÍTICO**
6. **Descripción del problema:** La función `getSecret()` está definida como `const getSecret = () => process.env.JWT_SECRET || config.JWT_SECRET || '';`.
7. **Explicación de por qué representa un problema:** Si la variable de entorno `JWT_SECRET` no está configurada, el servidor firma y verifica tokens usando una cadena vacía o falla de manera silenciosa en lugar de detener el arranque.
8. **Consecuencias que podría ocasionar:** Falsificación trivial de tokens JWT con privilegios de `Administrador`.
9. **Solución recomendada:** Validar que `JWT_SECRET` esté definido y tenga al menos 32 caracteres; en caso contrario, lanzar un error explícito.
10. **Ejemplo de código corregido (`Backend/src/config/jwt.js`):**

```
const getSecret = () => {
  const secret = process.env.JWT_SECRET || config.JWT_SECRET;
  if (!secret || secret.trim().length < 32) {
    throw new Error('JWT_SECRET no está configurado o es demasiado
corto.');
  }
  return secret;
};
```

AUD-003 — Exposición pública sin autenticación de imágenes de instalaciones y escritura síncrona en disco

1. **Identificador del problema:** `AUD-003`
2. **Archivo donde se encuentra:** `/var/www/html/Inventario_Travelnet/Backend/app.js` y `/var/www/html/Inventario_Travelnet/Backend/src/controller/imagen_InstalacionController.js`
3. **Línea aproximada del error:** `app.js` línea 42; `imagen_InstalacionController.js` líneas 27–36
4. **Categoría del problema:** Error de seguridad y Problema de rendimiento (Error confirmado)
5. **Nivel de gravedad:** **ALTO**

6. **Descripción del problema:** En `app.js` (línea 42), `app.use(express.static(path.join(__dirname, 'public')))` se registra antes del middleware de autenticación JWT (líneas 44–63). En `imagen_InstalacionController.js`, las imágenes se guardan en `public/uploads/evidencia_${Date.now()}.jpg` usando funciones síncronas `fs.mkdirSync` y `fs.writeFileSync`.
7. **Explicación de por qué representa un problema:** Las imágenes de instalación tienen impresas las coordenadas GPS exactas, la dirección física del cliente y el nombre del técnico. Al estar en `public/uploads` antes del middleware JWT y con nombres basados en timestamp, cualquier persona sin iniciar sesión puede descargarlas. Además, `fs.writeFileSync` bloquea el Event Loop de Node.js.
8. **Consecuencias que podría ocasionar:** Exposición de información privada de ubicación de clientes y bloqueo del servidor durante la subida de fotografías.
9. **Solución recomendada:** Utilizar `fs.promises.writeFile`, generar nombres con `crypto.randomUUID()` y servir las evidencias únicamente tras validar el token JWT.
10. **Ejemplo de código corregido (`imagen_InstalacionController.js`):**

```
const crypto = require('crypto');
const fsPromises = require('fs/promises');

if (req.body.Ruta_Imagen && /^data:image\/(jpeg|png|
webp);base64,/.test(req.body.Ruta_Imagen)) {
    const base64Data = req.body.Ruta_Imagen.replace(/^data:image\/\w+;base64,/, '');
    const nombreArchivo = `evidencia_${crypto.randomUUID()}.jpg`;
    const dirUploads = path.join(__dirname, '../../public/uploads');
    await fsPromises.mkdir(dirUploads, { recursive: true });
    const rutaFisica = path.join(dirUploads, nombreArchivo);
    await fsPromises.writeFile(rutaFisica, Buffer.from(base64Data, 'base64'));
    req.body.Ruta_Imagen = `/uploads/${nombreArchivo}`;
}
```

AUD-004 — Vulnerabilidad Cross-Site Scripting (DOM XSS) en la generación de tickets HTML

1. **Identificador del problema:** `AUD-004`
2. **Archivo donde se encuentra:** `/var/www/html/Inventario_Travelnet/Frontend/src/app/features/pago/pages/lista-pago/lista-pago.ts` y `.../insertar-pago/insertar-pago.ts`
3. **Línea aproximada del error:** `lista-pago.ts` líneas 201–265, 315–398, 400–488, 548–661; `insertar-pago.ts` líneas 199–312
4. **Categoría del problema:** Error de seguridad (Error confirmado)
5. **Nivel de gravedad:** **ALTO**
6. **Descripción del problema:** Los métodos `imprimirTicketCorte`, `imprimirMensualidadIndividual`, `imprimirTicket` e `imprimirTicketGuardado` interpolan directamente campos como `Nombre_Cliente`, `Concepto`, `Localidad`, `Plan` y `autorizador` dentro de un string HTML y lo escriben mediante `ventana.document.write(ticketHTML)` en una ventana

emergente que ejecuta scripts.

7. **Explicación de por qué representa un problema:** Al no escapar caracteres especiales HTML (<, >, ", ', &), cualquier etiqueta HTML o script almacenado en el nombre de un cliente o concepto de pago se ejecutará en el navegador con acceso al `localStorage` del dominio.
8. **Consecuencias que podría ocasionar:** Robo del token JWT de sesión y ejecución de operaciones no autorizadas desde el navegador del cajero o administrador.
9. **Solución recomendada:** Sanitizar/escapar todas las variables dinámicas antes de interpolarlas en el HTML del ticket.
10. **Ejemplo de código corregido:**

```
private escapeHtml(valor: unknown): string
{
  return String(valor ?? '')
    .replace(/&/g, '&amp;')
    .replace(/</g, '&lt;')
    .replace(/>/g, '&gt;')
    .replace(/"/g, '&quot;')
    .replace(/'/g, '&#39;');
}
```

> **AUD-005 — Falta de propagación de req.user en JWT y autorización de Corte de Caja validada solo en Frontend**

1. **Identificador del problema:** `AUD-005`
2. **Archivo donde se encuentra:** `/var/www/html/Inventario_Travelnet/Backend/app.js` y `/var/www/html/Inventario_Travelnet/Frontend/src/app/features/pago/pages/lista-pago/lista-pago.ts`
3. **Línea aproximada del error:** `app.js` líneas 53–57; `lista-pago.ts` líneas 163–186 y 731–769
4. **Categoría del problema:** Error de seguridad y arquitectónico (Error confirmado)
5. **Nivel de gravedad:** **ALTO**
6. **Descripción del problema:** El middleware JWT en `app.js` verifica el token y selecciona el pool de base de datos (`req.db = getDbPool(decoded.rol)`), pero no asigna `req.user = decoded`. Como resultado, los controladores confían en el `UsuarioId` enviado en `req.body`. Además, cuando un usuario `Mostrador` realiza un corte de caja en `lista-pago.ts`, la validación de las credenciales del Administrador ocurre en el Frontend y al Backend solo se envía el nombre del autorizador en texto (`Autorizador: usuarioAdmin.Nombre`).
7. **Explicación de por qué representa un problema:** Un usuario autenticado puede manipular el cuerpo JSON de la petición HTTP para registrar pagos, cancelaciones o cortes de caja a nombre de otro `UsuarioId` o indicar cualquier `Autorizador` sin que el Backend verifique la firma o contraseña del administrador.
8. **Consecuencias que podría ocasionar:** Inconsistencia en la trazabilidad y auditoría de cobros y cortes de caja.
9. **Solución recomendada:** Asignar `req.user = decoded` en `app.js`, usar `req.user.id` en los

controladores y verificar las credenciales o token del autorizador en el servidor al registrar un corte.

10. **Ejemplo de código corregido (`Backend/app.js`):**

```
const decoded = jwt.verify(token,
jwtConfig.getSecret());
req.user = decoded;
req.db = getDbPool(decoded.rol);
return next();
```

AUD-006 — Error funcional en el cálculo de Corte de Caja y desfase de zona horaria (UTC vs Local)

1. **Identificador del problema:** `AUD-006`
2. **Archivo donde se encuentra:** `/var/www/html/Inventario_Travelnet/Frontend/src/app/features/pago/pages/lista-pago/lista-pago.ts` y `.../home/pages/home/home.ts`
3. **Línea aproximada del error:** `lista-pago.ts` líneas 133–157; `home.ts` líneas 87–97
4. **Categoría del problema:** Error funcional (Error confirmado)
5. **Nivel de gravedad:** **ALTO**
6. **Descripción del problema:**
   - `prepararCorteCaja()` filtra sobre `this.pagos()`, que proviene de `Pago.findAllDetallado` (consulta que devuelve 1 fila por instalación con únicamente su último pago `MAX(p2.PagoId)`). Si una instalación registra dos pagos el mismo día, el primero no se suma al corte.
   - La fecha del día se obtiene con `new Date().toISOString().split('T')[0]`, que usa horario **UTC**. En México (UTC-6), después de las 18:00 hrs locales, `hoyISO` salta al día siguiente.
   - No se verifica si los pagos ya fueron incluidos en un corte de caja anterior del mismo día.
7. **Explicación de por qué representa un problema:** Descuadra el monto calculado del corte frente al efectivo real cobrado en ventanilla.
8. **Consecuencias que podría ocasionar:** Cortes de caja en cero o incompletos durante la tarde/noche y omisión de cobros cuando un cliente realiza más de una operación el mismo día.
9. **Solución recomendada:** Comparar fechas utilizando la fecha local del dispositivo y consultar los pagos efectivos del día que no estén ya asociados en `Corte_Pago`.
10. **Ejemplo de código corregido:**

```
private obtenerFechaLocalISO(fecha: Date = new Date()): string
{
  const anio = fecha.getFullYear();
  const mes = String(fecha.getMonth() + 1).padStart(2, '0');
  const dia = String(fecha.getDate()).padStart(2, '0');
  return `${anio}-${mes}-${dia}`;
}
```

AUD-007 — Sobrescritura forzada TarifaId = null al actualizar una instalación

1. **Identificador del problema:** `AUD-007`
2. **Archivo donde se encuentra:** `/var/www/html/Inventario_Travelnet/Frontend/src/app/features/instalacion/pages/actualizar-instalacion/actualizar-instalacion.ts` y `Backend/src/controller/instalacionController.js`
3. **Línea aproximada del error:** `actualizar-instalacion.ts` línea 146; `instalacionController.js` línea 45
4. **Categoría del problema:** Error funcional (Error confirmado)
5. **Nivel de gravedad:** **ALTO**
6. **Descripción del problema:** En `ActualizarInstalacion.guardarCambios()` se asigna `this.instalacion.TarifaId = null;` antes de enviar la actualización. Además, `instalacionController.update` combina `{ ...exist, ...req.body }`, provocando que cualquier actualización parcial (como guardar solo la ubicación GPS desde `lista-instalacion.ts`) vuelva a ejecutar un `UPDATE` de todas las columnas de la tabla `Instalacion`.
7. **Explicación de por qué representa un problema:** Borrar `TarifaId` obliga al modelo a buscar la tarifa por coincidencia de texto del nombre del plan, lo cual es frágil si el nombre del plan fue modificado.
8. **Consecuencias que podría ocasionar:** Pérdida de la tarifa asignada o actualizaciones innecesarias sobre todas las columnas de `Instalacion`.
9. **Solución recomendada:** Preservar `TarifaId` o vincular el selector del formulario al catálogo `GET /api/plan/tarifas`, y en `instalacionController.update` enviar únicamente `req.body`.
10. **Ejemplo de código corregido (`instalacionController.js`):**

```
const update = async (req, res) => {
    try {
        const exist = await Instalacion.findByPk(req.db, req.params.id);
        if (!exist) return res.status(404).json({ error: 'Instalación no encontrada' });

        const instalacionUpdated = await Instalacion.update(req.db, req.params.id,
req.body);
        res.json(instalacionUpdated);
    } catch (error) {
        res.status(500).json({ error: error.message });
    }
};
```

AUD-008 — Inconsistencia de mayúsculas en URLs de API y localhost:3000 hardcodeado en 12 servicios

1. **Identificador del problema:** `AUD-008`
2. **Archivo donde se encuentra:** `Backend/app.js` y los 12 servicios en `Frontend/src/app/features/**/service*.ts`

3. **Línea aproximada del error:** Líneas 10–25 de cada servicio en Angular; líneas 67–80 de `Backend/app.js`
4. **Categoría del problema:** Error de comunicación API y Mantenibilidad (Error confirmado)
5. **Nivel de gravedad:** **MEDIO**
6. **Descripción del problema:** `Backend/app.js` monta `/api/OLT`, `/api/torre` y `/api/corteCaja`, mientras que en Angular se invocan `/api/olt` (en `InstalacionService`), `/api/Torre` (en `TorreService`) y `/api/CorteCaja` (en `CorteCajaService` y `PagoService`). Además, todos los servicios tienen escrito de forma fija `http://localhost:3000/api/...`.
7. **Explicación de por qué representa un problema:** Impide consumir el Backend desde dispositivos móviles en la misma red local o en un servidor de producción sin modificar 12 archivos distintos, y puede causar errores `404` detrás de proxies sensibles a mayúsculas/minúsculas.
8. **Consecuencias que podría ocasionar:** Fallos de comunicación HTTP en despliegues reales o pruebas desde teléfonos de instaladores.
9. **Solución recomendada:** Definir `apiUrl` en un archivo de entorno centralizado y estandarizar el casing de todas las rutas.
10. **Ejemplo de código corregido (`src/environments/environment.ts`):**

```
export const environment = {
  production: false,
  apiUrl:
'http://localhost:3000/api'
};
```

AUD-009 — Falta de validación de idempotencia (Uuid_local) en Instalacion.create e Imagen_Instalacion.create

1. **Identificador del problema:** `AUD-009`
2. **Archivo donde se encuentra:** `/var/www/html/Inventario_Travelnet/Backend/src/model/instalacion.js` y `.../Imagen_Instalacion.js`
3. **Línea aproximada del error:** `instalacion.js` líneas 148–208; `Imagen_Instalacion.js` líneas 24–36
4. **Categoría del problema:** Error funcional / Base de datos (Error confirmado)
5. **Nivel de gravedad:** **MEDIO**
6. **Descripción del problema:** A diferencia de `Pago.create`, los métodos `Instalacion.create` e `Imagen_Instalacion.create` no comprueban si ya existe un registro con el mismo `Uuid_local` antes de ejecutar `INSERT`.
7. **Explicación de por qué representa un problema:** Las tablas `Instalacion` e `Imagen_Instalacion` tienen restricción `UNIQUE (Uuid_local)`. Si un reintento de sincronización envía un `Uuid_local` que ya se insertó previamente, MariaDB lanzará error de clave duplicada (`ER_DUP_ENTRY`).
8. **Consecuencias que podría ocasionar:** Registros atascados en la cola de sincronización offline de IndexedDB.

9. **Solución recomendada:** Comprobar previamente si `Uuid_local` ya existe y retornar el registro existente de forma idempotente.
10. **Ejemplo de código corregido (`instalacion.js`):**

```
if (Uuid_local) {
  const dup = await db.query('SELECT InstalacionId FROM Instalacion WHERE Uuid_local = ? LIMIT 1', [Uuid_local]);
  if (dup.length > 0) {
    return await Instalacion.findByPk(db, dup[0].InstalacionId);
  }
}
```

AUD-010 — Guard e Interceptor de Autenticación no validan expiración del JWT ni respuestas 401

1. **Identificador del problema:** `AUD-010`
2. **Archivo donde se encuentra:** `/var/www/html/Inventario_Travelnet/Frontend/src/app/shared/auth.guard.ts`, `shared/interceptors/auth.interceptor.ts` y `features/usuario/services/usuario.ts`
3. **Línea aproximada del error:** `usuario.ts` líneas 24–26; `auth.interceptor.ts` líneas 3–14
4. **Categoría del problema:** Error de comunicación API y Seguridad (Error confirmado)
5. **Nivel de gravedad:** **MEDIO**
6. **Descripción del problema:** `estaAutenticado()` únicamente comprueba `!! localStorage.getItem('token')` sin verificar el claim `exp` del JWT, y `authInterceptor` no intercepta respuestas HTTP `401 Unauthorized`.
7. **Explicación de por qué representa un problema:** Cuando el token expira tras 8 horas, el usuario continúa navegando por la aplicación; los servicios con soporte offline capturan el error `401` en su `catchError` como si fuera "falta de red" y muestran datos antiguos de IndexedDB sin redirigir al login.
8. **Consecuencias que podría ocasionar:** Operación con sesión vencida, visualización de datos obsoletos y fallos al intentar guardar registros.
9. **Solución recomendada:** Verificar la fecha de expiración del token en `estaAutenticado()` y redirigir a `/login` en `authInterceptor` al recibir un código `401`.
10. **Ejemplo de código corregido (`auth.interceptor.ts`):**

```
return next(peticion).pipe(
  catchError((err: HttpErrorResponse) => {
    if (err.status === 401 && !req.url.endsWith('/api/usuarios/login'))
{
      localStorage.removeItem('token');
      localStorage.removeItem('usuario');
      router.navigate(['/login']);
    }
    return throwError(() => err);
  })
```

);

AUD-011 — Manejador global de errores en Express devuelve HTML y controladores exponen error.message con código 500

1. **Identificador del problema:** `AUD-011`
2. **Archivo donde se encuentra:** `/var/www/html/Inventario_Travelnet/Backend/app.js` y `Backend/src/controller/*.js`
3. **Línea aproximada del error:** `app.js` líneas 82–91
4. **Categoría del problema:** Error arquitectónico y de comunicación API (Error confirmado)
5. **Nivel de gravedad:** **MEDIO**
6. **Descripción del problema:** El middleware final de errores en `app.js` ejecuta `res.render('error')` devolviendo HTML en vez de JSON. Además, los controladores devuelven siempre `res.status(500).json({ error: error.message })` incluso cuando el error es causado por datos de entrada faltantes o inválidos (que deberían responder `400 Bad Request`).
7. **Explicación de por qué representa un problema:** En `InsertarPago.guardarPago()` (línea 157), el Frontend guarda el pago en la cola offline si recibe `err.status >= 500`. Si el Backend devuelve `500` por un dato inválido, el pago erróneo se guarda en IndexedDB como pendiente de sincronizar.
8. **Consecuencias que podría ocasionar:** Encolamiento de datos inválidos en IndexedDB y exposición de mensajes internos de SQL al usuario final.
9. **Solución recomendada:** Responder siempre JSON en `app.js` y diferenciar errores de validación (`400`) de fallos internos del servidor (`500`).
10. **Ejemplo de código corregido (`Backend/app.js`):**

```
app.use(function(err, req, res, next) {
  const status = err.status || 500;
  res.status(status).json({
    error: status === 500 ? 'Error interno del servidor' :
err.message
  });
});
```

AUD-012 — Sobrecarga de peticiones en Home, llamadas manuales a ngOnInit() y falta de bloqueo en SyncService

1. **Identificador del problema:** `AUD-012`
2. **Archivo donde se encuentra:** `Frontend/src/app/features/home/pages/home/home.ts`, `.../pago/pages/lista-pago/lista-pago.ts` y `.../pago/services/sync.service.ts`
3. **Línea aproximada del error:** `home.ts` líneas 69–135; `lista-pago.ts` líneas 308, 312, 520; `sync.service.ts` líneas 18–32
4. **Categoría del problema:** Problema de rendimiento y mantenibilidad (Error confirmado)
5. **Nivel de gravedad:** **MEDIO**

6. **Descripción del problema:**
    - `Home.cargarMetricas()` descarga todas las filas de 6 tablas completas en peticiones separadas solo para calcular `.length` y usa un `setTimeout(..., 1200)` fijo para apagar el indicador de carga.
    - `ListaPago` llama directamente a `this.ngOnInit()` en tres lugares distintos para recargar la tabla.
    - `SyncService.sincronizarTodo()` carece de una bandera de exclusión mutua (`sincronizando`), permitiendo ejecuciones concurrentes si el evento `online` se dispara repetidamente.
7. **Explicación de por qué representa un problema:** Degrada el tiempo de carga inicial conforme crece el volumen de datos y puede causar condiciones de carrera durante la sincronización offline.
8. **Consecuencias que podría ocasionar:** Mayor consumo de memoria/red en el arranque y peticiones duplicadas de sincronización.
9. **Solución recomendada:** Extraer un método `cargarPagos()` en `ListaPago`, añadir una bandera booleana `private sincronizando = false;` en `SyncService` y usar `forkJoin` en `Home`.
10. **Ejemplo de código corregido (`sync.service.ts`):**

```
private sincronizando = false;

async sincronizarTodo() {
  if (this.sincronizando) return;
  this.sincronizando = true;
  try {
    await this.sincronizarClientes();
    await
this.sincronizarInstalaciones();
    await this.sincronizarPagos();
  } finally {
    this.sincronizando = false;
  }
}
```

AUD-013 — Dependencias erróneas en package.json e importaciones/código muerto

1. **Identificador del problema:** `AUD-013`
2. **Archivo donde se encuentra:** `Backend/package.json` (línea 12), `Frontend/package.json` (línea 23), `Backend/routes/Usuario.js` (línea 4), `Backend/src/controller/clienteController.js` (línea 2), `Frontend/.../insertar-instalacion.ts` (línea 1) y `Frontend/.../lista-pago.ts` (líneas 771–784)
3. **Línea aproximada del error:** Ver detalle de archivos
4. **Categoría del problema:** Problema de mantenibilidad / Incumplimiento de estándares (Error confirmado)
5. **Nivel de gravedad:** **BAJO**

6. **Descripción del problema:**
    - `Backend/package.json` incluye el paquete `"corns": "^0.0.0"` (error tipográfico al instalar `cors`).
    - `Frontend/package.json` incluye `"jsonwebtoken": "^9.0.3"`, librería exclusiva de Node.js que no se usa en el Frontend.
    - Existen importaciones sin uso (`const { route } = require('./users')` en `routes/Usuario.js`, `jwt` en `clienteController.js`, `Inject` en `insertar-instalacion.ts`) y el método duplicado sin uso `obtenerOcupacion()` en `lista-pago.ts`.
7. **Explicación de por qué representa un problema:** Introduce dependencias innecesarias en `node_modules` y reduce la limpieza del código.
8. **Consecuencias que podría ocasionar:** Confusión durante el mantenimiento y alertas en auditorías de dependencias NPM.
9. **Solución recomendada:** Desinstalar `corns` en Backend y `jsonwebtoken` en Frontend, y remover las importaciones y funciones no utilizadas.
10. **Ejemplo de código corregido (Comandos NPM):**

```
cd Backend && npm uninstall corns
cd ../Frontend && npm uninstall
jsonwebtoken
```

5. Plan de Corrección Priorizado

1. **Fase 1 — Seguridad Crítica y Alta (Inmediata):**
    - Actualizar `.gitignore` para proteger `**/.env`, `contraseñas.txt` y `PLAN_INSERTS_HASH.txt` (`AUD-001`).
    - Validar obligatoriedad de `JWT_SECRET` en `Backend/src/config/jwt.js` (`AUD-002`).
    - Escapar variables dinámicas en la generación de tickets HTML (`AUD-004`).
    - Propagar `req.user` en `Backend/app.js` y migrar la escritura de imágenes a `fs.promises.writeFile` con nombres UUID (`AUD-003`, `AUD-005`).
2. **Fase 2 — Estabilidad Funcional y Comunicación API (Corto Plazo):**
    - Corregir el cálculo de fecha local y filtrado de pagos en `prepararCorteCaja()` y `Home` (`AUD-006`).
    - Conservar `TarifaId` en `ActualizarInstalacion` y evitar sobrescritura masiva en `instalacionController.update` (`AUD-007`).
    - Añadir verificación de `Uuid_local` en `Instalacion.create` e `Imagen_Instalacion.create` (`AUD-009`).
    - Validar expiración del token JWT y manejar respuestas HTTP `401` en `authInterceptor` (`AUD-010`).
    - Convertir el manejador global de errores de Express a JSON (`AUD-011`).
3. **Fase 3 — Arquitectura, Rendimiento y Mantenibilidad (Mediano Plazo):**

- Centralizar la URL base del API en `environments` y homogeneizar mayúsculas/minúsculas en rutas (`AUD-008`).
- Agregar bloqueo de concurrencia en `SyncService`, optimizar la carga de métricas en `Home` y reemplazar las llamadas manuales a `this.ngOnInit()` (`AUD-012`).
- Consolidar los modelos duplicados en una carpeta compartida o en sus respectivas features, mover `auth.guard.ts` y `auth.interceptor.ts` a `core/`, y limpiar dependencias/imports sin uso (`AUD-013`).